\t tab\n space .removeprefix()
Deleteprefix .square=[value**2 for value in range(1,n)] Create nested list matrix = [list(map(int, input().split())) for _ in range(5)] list[sth]output sth's index

STRING SLICING

s = "abcdefgh" s[3:6]=def s[3:6:2]=df s[::-1]=hgfedcba s[4:1:-2]=ec If there are spaces in the string, the spaces will occupy positions when slicing. s="ABC d3f" s[4:0:-1]="d CB" When using join() on a string, first convert to a list print(" ".join([verb] * 5))

BREAK STATEMENT

```
mysum = 0
for i in range(5, 11, 2):
        mysum += i
        if mysum == 5:
                break
print(mysum)
```

guess_and_check:quare root with while loop

```
guess = 0
active = False
x = int(input("Enter a positive integer: ")
if x < 0:
        active = True
while guess**2 < x:
        guess = guess + 1
if guess**2 == x:
        print(f"Square root of {x} is {guess}")
else:
        print(f"{x} is not a perfect square")
        if active:
                print(f"Just checking…did you
mean {x} ?")
```

cube root (NEGATIVE INTERGER)

```
cube = int(input("Enter an integer: "))
for guess in range(abs(cube)+1):
        if guess**3>=abs(cube):
                break
if guess**3 != abs(cube):
        print(cube,"is not a perfect cube")
else:
        if cube < 0:
                guess = -guess
        print("Cube root
 of"+str(cube)+"is"+str(guess))
```

FROM INT_DECIMAL TO BINARY

```
if num < 0:
        is_neg = True
        num = abs(num)
else:
        is_neg = False
result = ''
if num == 0:
        result = '0'
while num > 0:
        result = str(num%2) + result
        num = num//2
if is_neg:
        result = '-' + result
```

FROM FRACTION TO BINARY

```
x =0.625
p = 0
while ((2**p)*x)%1 != 0:
        print('Remainder = ' + str((2**p)*x -
int((2**p)*x)))
        p += 1
num = int(x*(2**p))
result = ''
if num == 0:
        result = '0'
while num > 0:
        result = str(num%2) + result
        num = num//2
for i in range(p- len(result)):
        result = '0' + result
result = result[0:-p] + '.' + result[-p:]
print('The binary representation of the
decimal ' + str(x) + ' is ' + str(result))
```

Never use == to test floats

APPROXIMATION ALGORITHM

```
x = 54321
epsilon = 0.01
numGuesses = 0
guess = 0.0
increment = 0.0001# Reset smaller increment
means more values will be checked but
Program will be slower
while abs(guess**2- x) >= epsilon and
guess**2 <= x:
        guess += increment
        numGuesses += 1
print('numGuesses =', numGuesses)
if abs(guess**2- x) >= epsilon:
        print('Failed on square root of', x)
else:
        print(guess, 'is close to square root of',
x)
```

BISECTION SEARCH

```
x = 54321
epsilon = 0.01
num_guesses = 0
if x >= 1:
        low = 1.0
        high = x
else:
        low = x
        high = 1.0
guess = (high + low)/2.0
while abs(guess**2- x) >= epsilon:
        if guess**2 < x :
                low = guess
        else:
                high = guess
        guess = (high + low)/2.0
        num_guesses += 1
print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```

NEWTON-RAPHSON ROOT FINDER

```
epsilon = 0.01
k = 24.0
guess = k/2.0 woddd
num_guesses = 0
while abs(guess*guess - k) >= epsilon:
        num_guesses += 1
        guess = guess - (((guess**2) -
        k)/(2*guess))
print('num_guesses = ' + str(num_guesses))
print('Square root of ' + str(k) + ' is about ' +
str(guess))
```

Positional arguments MUST follow the sequence, Keyword arguments needn't

Inside a function, can access a variable defined outside but cannot modify a variable defined outside (can by using global variables, but frowned upon)

FUNCTION AS A PARAMETER

```
def calc(op, x, y):
        return op(x,y)
def add(a,b):
        return a+b
def div(a,b):
        if b != 0:
                return a/b
        print("Denominator was 0.")
print(calc(add, 2, 3)) #5

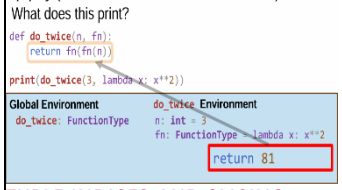
def func_a():
        print('inside func_a')
def func_b(y):
        print('inside func_b')
        return y
def func_c(f, z):
        print('inside func_c')
        return f(z)
print(func_a()) # inside func_a None
print(5 + func_b(2))#inside func_b 7
print(func_c(func_b, 3))#insidefunc_c
insidefuc_b 3
```

FUNCTIONS CAN RETURN FUNCTIONS

```
def make_prod(a):
        def g(b):
                return a*b
        return g
val = make_prod(2)(3)
print(val)#6
```

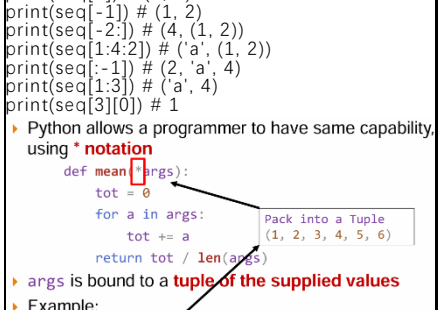
LAMBDA FUNCTIONS

```
apply(is_even, 10)
apply(lambda x: x%2 == 0, 10)
```



TUPLE INDICES AND SLICING

```
seq = (2, "a", 4, (1, 2))
print(len(seq)) # 4
print(seq[3]) # (1, 2)
print(seq[-1]) # (1, 2)
print(seq[-2:]) # (4, (1, 2))
print(seq[1:4:2]) # ('a', (1, 2))
print(seq[:-1]) # (2, 'a', 4)
print(seq[1:3]) # ('a', 4)
print(seq[3][0]) # 1
```




```
L1 = ['re'] L2 = ['mi'] L3 = ['do']
L4 = L1 + L2 #['re', 'mi']
L3.append(L4) #['do',['re', 'mi']]
L = L1.append(L3)#None
```

STRING TO LISTS

s.split() no parameter:split by space

LISTS TO STRINGS

Use ''.join(L) to turn a list of strings into a bigger string

```
L =['a','b','c']#LIST
A = ''.join(L) #"abc
B ='_'.join(L) #"a_b_c"
C=''.join([1,2,3]) error
C=' '.join(['1','2','3'] "a b c"
```

FUNCTION CAN CHANGE SEQUENCES

```
L=[9,6,0,3]
a = sorted(L) → returns a new sorted list,
does not mutate L
b = L.sort() → mutates L to be [0,3,5,6,9]
and returns None
L.reverse() → mutates L to be [9,6,5,3,0] and
returns None
```

FIND INDEX IN LIST

```
Index = list.index(sth)
Print(index)
```

FIND ALL INDEX OF A STH IN LIST

```
My_list = ['a', 'b', 'a', 'c']
Target = 'a'
Indices = [i for i, x in enumerate(my_list) if x
== target]
Print(indices)#[0,2]
```

FIND ALL INDEX OF A STH IN LIST FROM A SPECIFIC INDEX

```
My_list = ['a', 'b', 'a', 'c']
Index = my_list.index('a',1) #2
Index = my_list.index('a', 1,3)#There are two
valid values, but index() will only return the
first one found, namely 1
```

FUNCTION EXTEND

Append all elements from another sequence (list, tuple, string, etc.) to the end of the list (unpacking only one level)

```
numbers = [1, 2, 3]
numbers.extend([4, 5])
print(numbers)    # [1, 2, 3, 4, 5]
chars = ['a', 'b']
chars.extend('hello')
print(chars)    #['a', 'b', 'h', 'e', 'l', 'l', 'o']
empty_list = []
empty_list.extend([1, 2, 3])
print(empty_list)    # [1, 2, 3]
L2 = [4,5,6]
L2.extend([[1,2],[3,4]])# [4,5,6,[1,2],[3,4]]
```

REMOVE ALL ELEMENTS IN A LIST

List.clear()#Doesn't make a new empty list

OPERATION ON LISTS: remove

```
L = [2,1,3,6,3,7,0]
L.remove(2)#[1,3,6,3,7,0]
Lremove(3)#[2,1,6,3,7,0]
Del(L[1])#[2,3,6,3,7,0]
A = L.pop()#a=0 and L=[2,1,3,6,3,7]
```

EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR (remenber elements will move to another index when loop the list!)

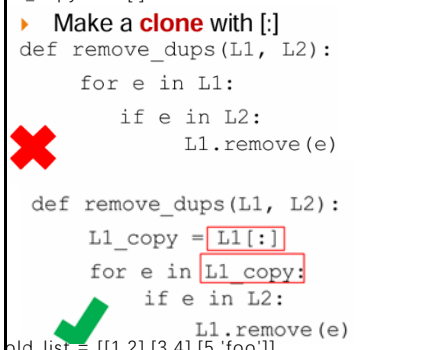
```
def remove_all(L, e):
        while e in L:
                L.remove(e)
```

MAKE A COPY OF THE LIST

```
Lcopy = L[:]
```

MUTATION AND ITERATION WITH CLONE

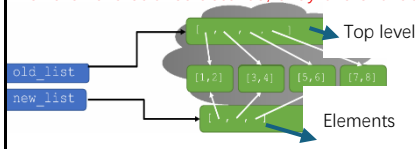
```
L1_copy = L1[:]
```




```
old_list = [[1,2],[3,4],[5,'foo']]
new_list = old_list
new_list[2][1] = 6# New list: [[1,2],[3,4],[5,6]]
Old list: [[1,2],[3,4],[5,6]](Assignment just
creates a new pointer to same object)
```

LIST SHALLOW COPY

```
import copy
Old_list = [1,2,3,4]
New_list = copy.copy(Old_list) #equivalent to
Lcopy = L[:]
old_list.append([7,8])#New list:
[[1,2],[3,4],[5,6]], Old list:
[[1,2],[3,4],[5,6],[7,8]](mutate top level
structures of one doesn't affect the clone)
old_list[1][1] = 9# New list: [[1,2],[3,9],[5,6]]
Old list: [[1,2],[3,9],[5,6]](change an element
in one of the sub-structures, they are shared)
```



LIST DEEP COPY

```
New_list = copy.deepcopy(old_list)
old_list[1][1] = 9# New list: [[1,2],[3,4],[5,6]]
Old list: [[1,2],[3,9],[5,6]]
```

are aliases

LIST COMPREHENSION

[expression for elem in iterable if test](test is optional)

```
def f(L):
        Lnew = []
        for e in L:
                if e%2 == 0:
                        Lnew.append(e**2)
        return Lnew
```

is equivalent to:

```
Lnew = [e**2 for e in L if e%2 == 0]
Def f(expr, old_list, test = lambda x:True):
        New_list = []
        For e in old_list
                If test(e):
                        New_list.append(expr(e))
        Return new_list
[[e,e**2] for e in range(4) if e%2==0]#[[1,1],
[3,9]]
```

HANDLING EXCEPTIONS

```
try:
        # do some potentially
        # problematic code
except:
        # do something to
        # handle the problem
def sum_digits(s):
        total = 0
        For char in s:
                try:
                        val = int(char)
                        total += val
                except:(there are nondigits)
                print("can't convert", char)
        return total
try:
        a = int(input("Tell me one number: "))
        b = int(input("Tell me another number:
"))    print("a/b = ", a/b)
        print("a+b = ", a+b)
except ValueError:
        print("Could not convert to a number.")
except ZeroDivisionError:
        print("Can't divide by zero")
        print("a/b = infinity")
        print("a+b =", a+b)
except:
        print("Something went very wrong.")
```

WHAT TO DO WITH EXCEPTIONS

1.fall silently
2.return an error value
3.stop execution, signal error condition(raise an exception)

```
def sum_digits(s):
        total = 0
        for char in s:
                try:
                        val = int(char)
                        total += val
                except:
                        raise ValueError("string
                contained a character")
```

ASSERTIONS

```
assert len(s) != 0, "s is empty
```

CREATING DICTIONARIES

```
Empty_dict={}
Empty_dict=dict()
grades = dict(math=95, science=88)

keys = ['a', 'b', 'c']
values = [1, 2, 3]
mapped = dict(zip(keys, values))

squares = {x: x*x for x in range(5)}
```

DICTIONARY LOOKUP

```
grades = {'Ana':'B', 'Matt':'A', 'John':'B', }
grades['John']#'B'
grades['Grace']#KeyError
print(grades.get('Ana'))# 'B'
print(grades.get('Kay'))# None
```

```
print(grades.get('Kay', 'Not found'))#'Not found'
```

DICTIONARY OPERATIONS

```
Add an entry grades['Grace']='A'
grades.update({'cyh': 'A', 'klein': 'A'})
Change entry grades['Grace']='C'
Delete entry del(grades['Ana'])
key, value = grades.popitem()#('John','B')
Test if key in dictionary
'John' in grades#True
'Daniel' in grades#False
'B' in grades#False(only test key, not value)
Get an iterable that acts like a tuple of all keys
Print(grades.keys())#dict_keys(['Ana', 'Matt', 'John'])
list(grades.keys())#return['Ana', 'Matt', 'John']
Get an iterable that acts like a tuple of all dict values
Print(grades.values()) dict_values(['B', 'A', 'B',])
list(grades.values())#returns ['B', 'A', 'B']
Get an iterable that acts like a tuple of all items
Print(g())#dict_items([('Ana', 'B'), ('Matt', 'A'), ('John', 'B')])
list(grades.items())#returns [('Ana', 'B'), ('Matt', 'A'), ('John', 'B')]
```

Typical use is to **iterate over key,value tuple**

```
for k,v in grades.items():
    print(f"key {k} has value {v}")
```

key Ana
key Mat

CREATING A DICTIONARY

```
song="RAH RAH AH AH AH ROM MAH RO MAH MAH"
    def generate_word_dict(song):
    song_words=song.lower()
    words_list=song_words.split()
    word_dict={}
    for w in words_list:
        if w in word_dict:
            word_dict[w]+=1
        else:
            word_dict[w]=1
return word_dict#word_dict = {'rah':2, 'ah':3, 'rom':1, 'mah':3, 'ro':1}

def find_frequent_word(word_dict):
    words = []
    highest = max(word_dict.values())
    for k,v in word_dict.items():
        if v == highest:
            words.append(k)
    return (words, highest)#(['ah','mah'],3)
def occurs_often(word_dict,x):
    freq_list=[]
    word_freq_tuple=find_frequent_word(word_dict)
        while word_freq_tuple[1]>x:
        freq_list.append(word_freq_tuple)
        for word in word_freq_tuple[0]:
            del(word_dict[word])
        word_freq_tuple=find_frequent_word(word_dict)
return freq_list# [(['ah','mah'],3), (['rah'],2)]
```

RECURSION

```
Recursivestep If b != 1, a*b = a + a*(b-1) }
Basecase If b = 1, a*b = a
def mult_recur(a, b):
    if b == 1:
        return a
    else:
        return a + mult_recur(a, b-1)
```

HOW TO UNPACKE NESTED LIST

```
L = [[1,2],3,4,[[5,6],[7,8]]]
Def nested(L):
    New_L = []
    For e in L:
        If type(e) == list:
            New_L.extend(nested(e))
        Else:
            New_L.extend(e)
    Return new_L#[1,2,3,4,5,6,7,8]
```

REVERSE ALL ELEMENTS IN ALL SUBLISTS

```
def deep_rev(L):
    L2 = []
    for e in L[::-1]:
        if type(e) == list:
            L2.append(deep_rev(e))
        else:
            L2.append(e)
    return L2#[[[8,7],[6,5]],4,3,[2,1]]
```

FIBONACCI

```
Def fib(n):
    If n <= 1:
        Return 1
    Return fib(n-1) + fib(n-2)
```

BINARY SEARCH-RECURISON WAY

```
Def binary_search(a, target, l=0, r=None):
    If r is None:
        r = len(a)-1
    if l > r:
        return -1
    m = (l + r)//2
    if a[m] == target:
        return m
    if a[m]<target:
        return binary_search(a,target, m+1, r)
    else:
        return binary_search(a,target,l,m-1)
```

PROBLEM-lower triangular mutipulation

```
m = float(input())
n = int(m)

if n!=m or n<1 or n>9:

    print("Invalid")
else:
    for i in range(1, n+1):
        row = ''
        for j in range(1, i+1)
            row += f"{j}x{i}={i*j:2}" + "\t"
        print(row)
```

PROBLEM-Rhombus generator

```
h = int(input())
w = int(input())
c = input()
middle = h // 2
max_chars = 1+middle*w
for i in range(middle + 1):
    num_chars = 1 + i*w
    space = (max_chars – num_chars)//2
    print(" "*space + c*num_chars)
for i in range(middle – 1, -1, -1)
    num_chars = 1 + i*w
    space = (max_chars – num_chars)//2
    print(" "*space + c*num_chars)
```

OR:

```
For i in range(h):
    Distance = abs(i – middle)
    Num_chars = max_chars – distance*w
    Space = (max_chars – num_chars)//2
    Print(" " * space + c*num_chars)
```

PROBLEM-Palindromic substring counter

```
s = input()
n = len(s)
cnt = 0
for c in range(n):
    l = r = c
    while l>=0 and r<n and s[l] == s[r]:
        cnt += 1
        l -= 1
        r += 1
for c in range(n-1):
    l, r = c, c+1
    while l>=0 and r<n and s[l] == s[r]:
        cnt += 1
        l -= 1
        r += 1
print(cnt)
```

PROBLEM-Calculator with lambda functions

```
def get_op(op_name: str):
    if op_name == "add":
        return lambda x, y: x + y
    elif op_name == "sub":
        return lambda x, y: x - y
    elif op_name == "mul":
        return lambda x, y: x * y
    elif op_name == "div":
        return lambda x, y: x / y
    elif op_name == "pow":
        return lambda x, y: x ** y
    elif op_name == "max":
        return lambda x, y: x if x>=y else y
    elif op_name == "min":
        return lambda x, y: x if x<=y else y
def calc(op, x: float, y: float) -> float:
    return op(x, y)
```

Lambda 的应用场景

一种常见的用途是作为参数传递给其他函数

- 如内置函数 `map()`, `sorted()` 等（传入 Lambda 表达式用作操作的参照依据）

场景：有一个数字列表，我们想得到每个数字的平方组成的新列表。

- 一般写法：for 循环遍历每个数，进行平方并附加到一个新的列表
- **使用 `lambda` 的精简写法：**

```
numbers = [1, 2, 3, 4, 5]
squared_numbers = map(lambda x: x * x, numbers)
# map: 对可迭代对象 (number) 中的每个元素应用传入的 lambda 表达式，
# 并将其结果写到一个新对象（此处为列表）中的对应位置，最后返回这个新对象
print(list(squared_numbers))
```

DICTIONARY COMPREHENSION

Transform existing dictionary

```
person = {'name': 'alice', 'age': '30'}
uppercase = {k: v.upper() if isinstance(v, str) else v for k, v in person.items()}#{'name': 'ALICE', 'age': '30'}
```

swap keys and values

```
original = {'a': 1, 'b': 2, 'c': 3}
swapped = {v: k for k, v in original.items()}#{1: 'a', 2: 'b', 3: 'c'}
```

Merging Dictionaries

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
merged={**dict1, **dict2}#{'a': 1, 'b': 3, 'c': 4}
dict1.update(dict2)#dict1={'a': 1, 'b': 3, 'c': 4}
```

Useful Dictionary Methods

```
defaults = dict.fromkeys(['name', 'age', 'city'], 'unknown')# {'name': 'unknown', 'age': 'unknown', 'city': 'unknown'}
if person: # True if not empty
    print("Dict has items")
```